



Programmation Orientée Objets

(II 1)

Les classes en c++

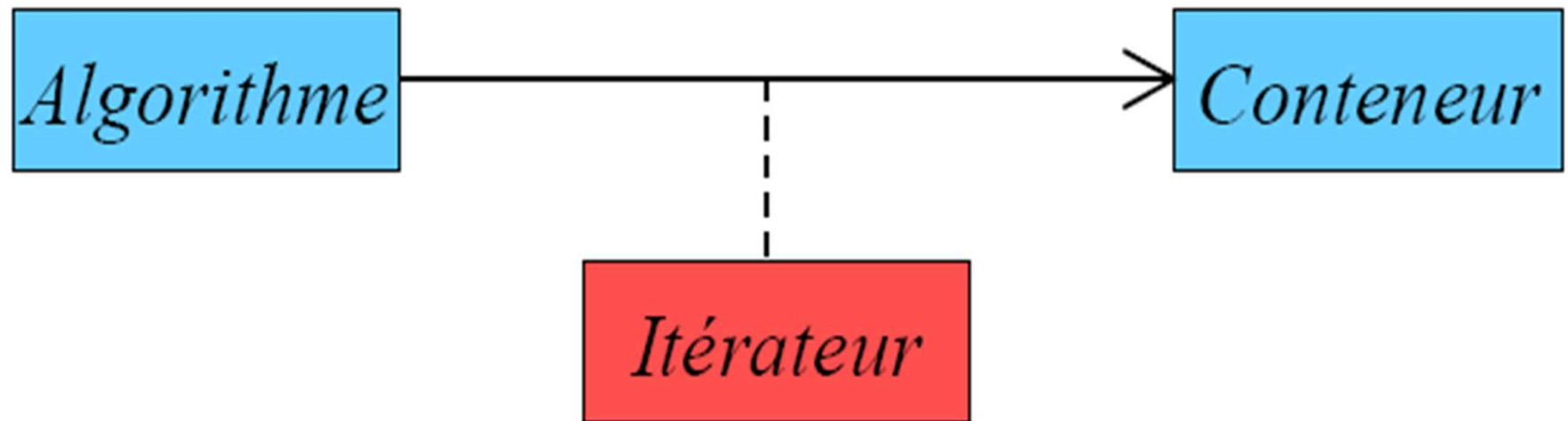


Qu'est-ce que STL?

STL = **S**andard **T**emplate **L**ibrary

- Il s'agit d'une bibliothèque générique qui fournit des solutions pour gérer un ensemble de données en utilisant des algorithmes efficaces
- Du point de vue du programmeur, STL fournit un groupe de classes répondant à divers besoins.
- Tous les composants de STL sont des **templates**

Les composants STL



La programmation générique

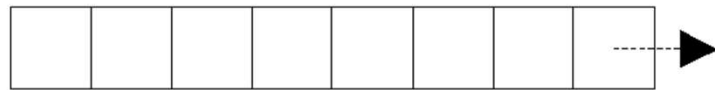
Il s'agit de connecter des *algorithmes* génériques
à des *structures de données* génériques
en utilisant des *itérateurs*.

Les conteneurs

- Les conteneurs servent à stocker des **collections d'objets** d'un même **type T**.
- STL fournit différentes sortes de conteneurs pour combler différents besoins. Il existe deux types de conteneurs:
 - Les **conteneurs séquentiels**, qui conservent la collection d'objets, sous un format strictement linéaire;
 - Les **conteneurs associatifs triés**, qui conservent la collection selon un ordre de tri basé sur une clé.

Les principaux conteneurs de STL

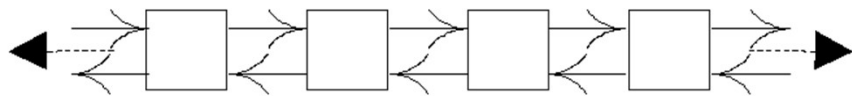
Conteneurs séquentiels



Vector

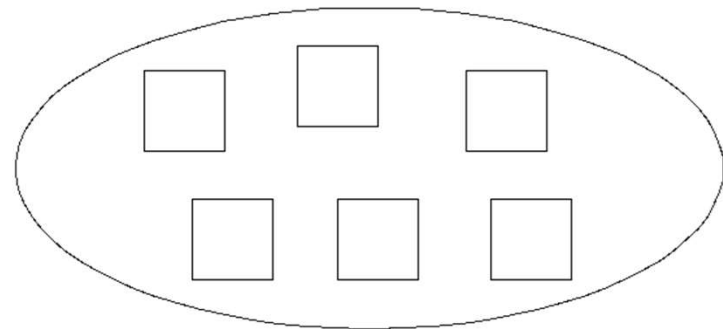


Deque

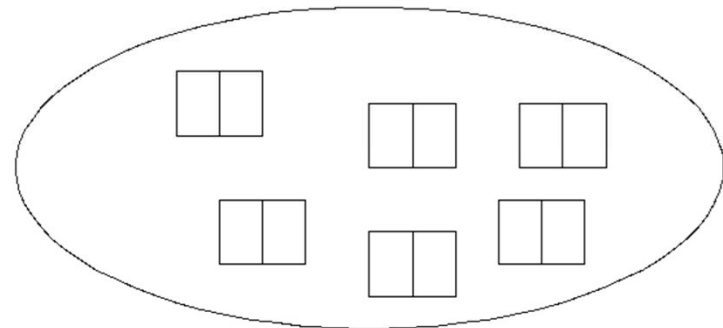


List

Conteneurs associatifs

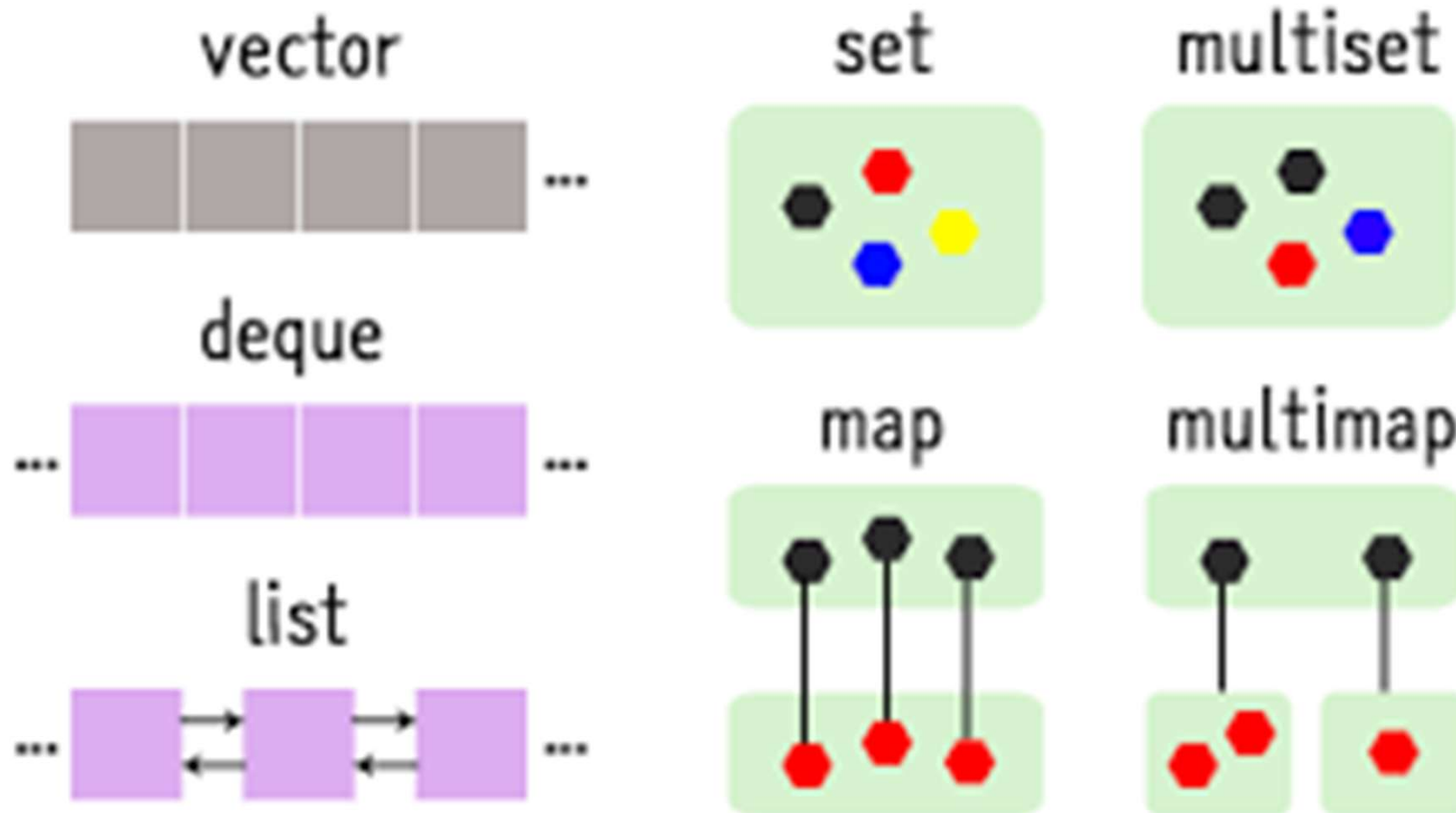


Set/Multiset



Map/Multimap

Les principaux conteneurs de STL



Conteneurs séquentiels

■ Le tableau C++ ordinaire:

■ Caractéristiques principales:

- ☑ Accès aléatoire par index en temps constant $O(1)$;
- ☑ Insertion et retrait lents au milieu du tableau;
- ☑ Taille ne pouvant être modifiée pendant l'exécution;

↩ Peut être utilisé avec les algorithmes de la STL comme tout autre conteneur de séquence.

■ Vector =

- Tableau dynamique
- Les éléments sont insérés à la fin du tableau

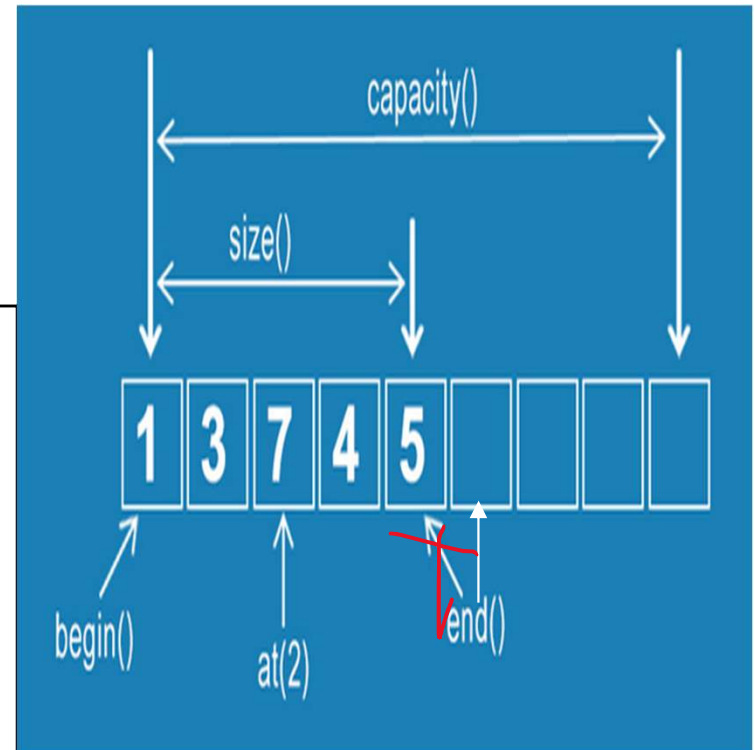


T a[n]

Conteneurs - vector

```
#include <vector>
#include <iostream>
using namespace std;

int main()
{
    vector<int> iv;
    cout<<"taille"<<iv.size()<<endl;
    cout<<"capacité"<<iv.capacity()<<endl;
    for( int i=0; i<5; ++i )
    {
        iv.push_back( i );
    }
    cout<<"taille"<<iv.size()<<endl;
    cout<<"capacité"<<iv.capacity()<<endl;
    cout<<"les éléments du vector:"<<endl;
    for( int i=0; i<iv.size(); ++i )
    {
        cout << iv[i] << endl;
    }
    cout<<"l'élément n°3= " << iv.at(3) << endl;
    iv.pop_back();
    cout<<"après suppression"<<endl;
    cout<<"taille"<<iv.size()<<endl;
    cout<<"capacité"<<iv.capacity()<<endl;
    cout<<"les éléments du vector:"<<endl;
    for( int i=0; i<iv.size(); ++i )
    {
        cout << iv[i] << endl;
    }
}
```



```
taille0
capacit00
taille5
capacit08
les 0l0ments du vector:
0
1
2
3
4
l'0l0ment n°3= 3
apr0s suppression
taille4
capacit08
les 0l0ments du vector:
0
1
2
3
```


Spécificité du conteneur Vector

- Les conteneurs séquentiels
 - Les éléments du conteneur ont une position qui dépend du temps et de l'endroit de l'insertion
 - La position est donc indépendante de la valeur de l'élément
- La taille d'un vector est le nombre d'éléments actuellement contenu dans le vector
- La capacité d'un vector est le nombre maximum d'éléments qui peuvent être contenus sans réallocation.
- La méthode **reserve()** est utilisée pour élargir la capacité
- La méthode **resize()** redimensionne le vector à **la taille** spécifiée (appelle le constructeur par défaut de chaque élément du tableau).

```

#include <vector>
#include <iostream>
using namespace std;
int main()
{
    vector<int> iv;    // vector a initialement 0 elements.
    iv.push_back(42); // Ajoute 42 à la fin du vector
    cout << "location: " << (void*) &iv[0] << endl;
    int* k = &iv[0]; // Attention ! Location peut changer !
    cout << "iv[0] = " << iv[0] << endl;
    cout << "*k = " << *k << endl;
    cout << "sz=" << iv.size() << " cap=" << iv.capacity();
    iv.reserve(1024); // augmente la capacité et non pas taille
    cout << "\nlocation: " << (void*) &iv[0] << endl;
    cout << "sz=" << iv.size() << " cap=" << iv.capacity();
    iv.resize(2048); // augmente taille (et capacité)
    cout << "\nlocation: " << (void*) &iv[0] << endl;
    cout << "sz=" << iv.size() << " cap=" << iv.capacity();
    cout << "\niv[0] = " << iv[0] << endl;
    cout << "*k = " << *k << endl; // danger! mauvaise valeur
}

```

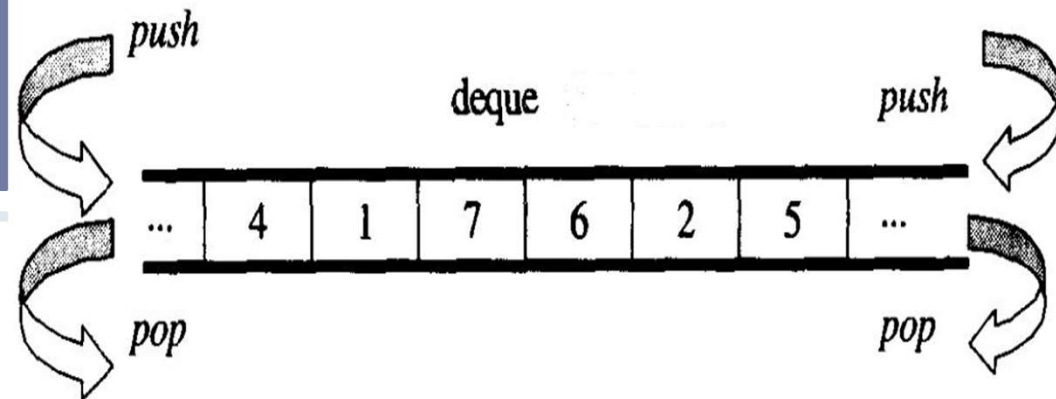
```

location: 0x4e7a440
iv[0] = 42
*k = 42
sz=1 cap=1
location: 0x4e86628
sz=1 cap=1024
location: 0x4e87630
sz=2048 cap=2048
iv[0] = 42
*k = 82288840

```

Conteneurs - deque

- Deque = Double-Ended Queue
- Tableau dynamique
- Les éléments sont insérés à la fin ou au début du tableau

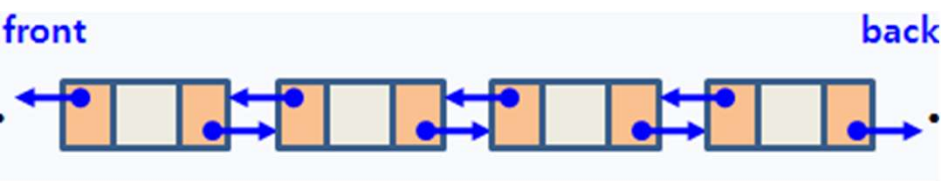


```
#include <deque>
#include <iostream>
using namespace std;
int main()
{
    deque<int> d;
    for(int i=0; i<10; ++i)
    {
        d.push_front( i );
    }
    for(int i=90; i<100; ++i )
    {
        d.push_back( i );
    }
    for(int i=0; i<d.size(); ++i)
    {
        cout << d[i] << endl;
    }
}
```

9
8
7
6
5
4
3
2
1
0
90
91
92
93
94
95
96
97
98
99

Conteneurs - list

- Liste doublement chaînée d'éléments
- Un élément pointe sur son prédécesseur et son successeur dans la liste
- Les éléments sont insérés n'importe où dans la liste



```
#include <list>
#include <iostream>
using namespace std;

int main()
{
    list<int> l;
    for( int i=1; i<20; ++i )
    {
        if( i % 2 == 0 )
            l.push_front(i);
        else
            l.push_back(i);
    }
    while( !l.empty() )
    {
        cout << l.front() << " ";
        l.pop_front();
        // supprime le 1er élément
    }
    cout << endl;
}
```

18 16 14 12 10 8 6 4 2 1 3 5 7 9 11 13 15 17 19

```

#include <list>
#include <iostream>
using namespace std;

int main()
{
    list<int> l;
    for( int i=1; i<20; ++i )
    {
        if( i % 2 == 0 )
            l.push_front(i);
        else
            l.push_back(i);
    }
    //18 16 14 12 10 8 6 4 2 1 3 5 7 9 11 13 15 17 19
    auto it=l.begin();
    advance(it, 3);
    l.insert(it, 12);
    /*for(auto it2 = l.begin(); it2 != l.end(); ++it2){ //entre /**/ Autre façon pour afficher le contenu d'une liste
        cout << *it2 << " ";
    }
    cout << endl;*/
    while( !l.empty() ) {
        cout << l.front() << " ";
        l.pop_front();
    }
    cout << endl;
}

```

18 16 14 12 12 10 8 6 4 2 1 3 5 7 9 11 13 15 17 19

Comparaisons

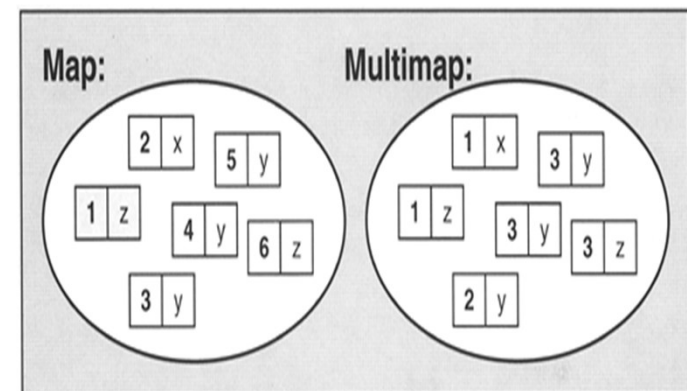
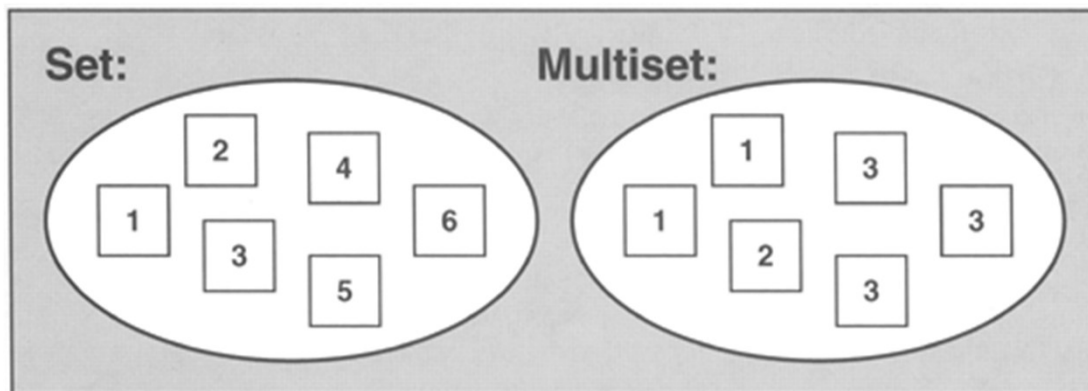
- Les `vector` et les `deque` permettent d'accéder rapidement à un élément de la liste (opérateur `[]`) contrairement aux `list`
- Un élément peut être inséré
 - n'importe où pour une `list`
 - à la fin ou au début pour un `deque`
 - à la fin seulement pour un `vector`
- Utiliser un `vector` est un bon choix lorsque:
 - on a besoin d'insérer/supprimer des éléments seulement à la fin
 - le conteneur doit être compatible au tableau C standard
- Utiliser un `deque` est un bon choix lorsque:
 - on a besoin d'insérer/supprimer des éléments seulement à la fin ou au début
 - le conteneur ne doit pas être compatible avec un tableau C standard
 - la taille maximum requise du conteneur n'est pas connue
- Utiliser une `list` est un bon choix lorsque:
 - on a besoin d'insérer/supprimer des éléments au milieu du conteneur
 - le conteneur ne doit pas être compatible avec un tableau C standard
 - la taille maximum requise du conteneur n'est pas connue

Comparaisons

- Un vector est un tableau dynamique qui ne se remplit qu'à la fin.
- Un deque est un tableau dynamique, découpé en blocs, qui peut s'étendre par les deux extrémités.
- Une liste est une liste doublement chaînée utilisant des pointeurs.
- Le vector:
 - + l'accès direct aux éléments est possible
 - + temps constant pour insertion et suppression à la fin
 - temps linéaire pour insertion et suppression ailleurs
 - redimensionnement demande une recopie complète du tableau - pas de `push_front()`, ni de `pop_front()`.
- Le deque :
 - + l'accès direct aux éléments est possible
 - + redimensionnement ne demande pas une recopie du tableau
 - + temps constant pour insertion et suppression au début et à la fin
 - temps linéaire pour insertion et suppression ailleurs
- La liste:
 - + temps constant pour insertion et suppression n'importe où
 - + l'insertion et la suppression n'invalident pas les itérateurs
 - + redimensionnement ne demande pas une recopie du tableau
 - + peut facilement être divisé, joint et inversé
 - l'accès direct aux éléments n'est pas possible

Conteneurs associatifs

- Les éléments du conteneur sont automatiquement triés lors de leur insertion selon un critère précis.
- Le critère de tri prend la forme d'une comparaison de la valeur de l'élément ou d'une clé spéciale définie pour l'élément.
- Les conteneurs associatifs sont généralement représentés sous forme d'arbre binaire
- `set`, `multiset`, `map` et `multimap` sont des conteneurs associatifs

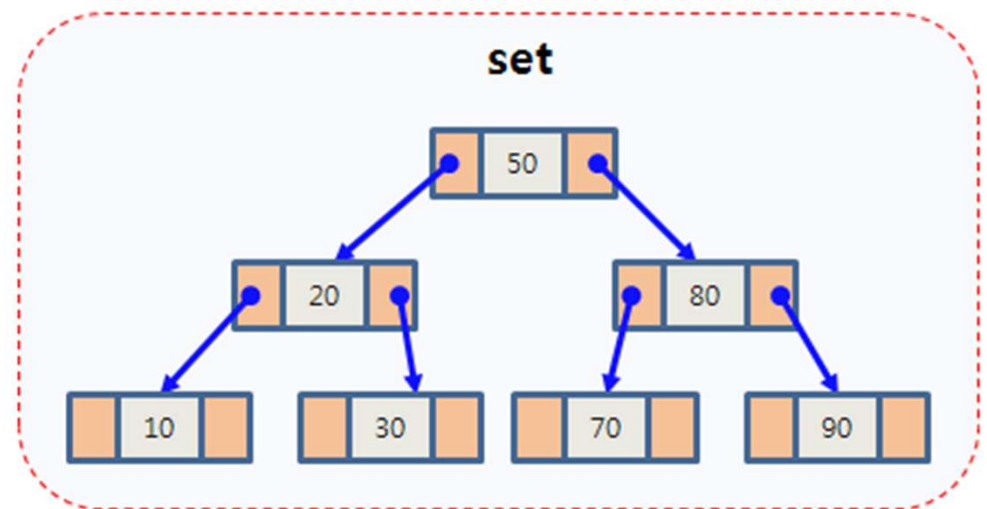


Conteneurs - set

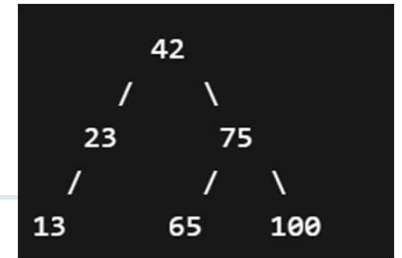
- Les éléments sont triés selon leur valeur. Chacun des éléments ne peut exister qu'une fois

```
#include <set>  
set<int> s;
```

```
s.insert( 50 );  
s.insert( 10 );  
s.insert( 20 );  
s.insert( 80 );  
s.insert( 30 );  
s.insert( 70 );  
s.insert( 90 );  
s.insert(10);
```



Conteneurs - set



```
#include <iostream>
#include <set>
using namespace std;
int main(){
    int desEntiers1[] = {75, 23, 65, 42, 13, 100}; // non ordonné
    int desEntiers2[] = {75, 23, 75, 23, 13}; // non ordonné avec des doublons
    set<int> ensemble1 (desEntiers1, desEntiers1+6); // the range (first,last)
    set<int> ensemble2 (desEntiers2, desEntiers2+5); // the range (first,last)
    cout << "L'ensemble set 1 contient :";
    for(set<int>::iterator it=ensemble1.begin(); it!=ensemble1.end(); ++it)
        {cout << ' ' << *it;}
    cout << endl;
    cout << "L'ensemble set 2 contient :";
    for(set<int>::iterator it=ensemble2.begin(); it!=ensemble2.end(); ++it)
        {cout << ' ' << *it;}
    cout << endl;
    return 0;
}
```

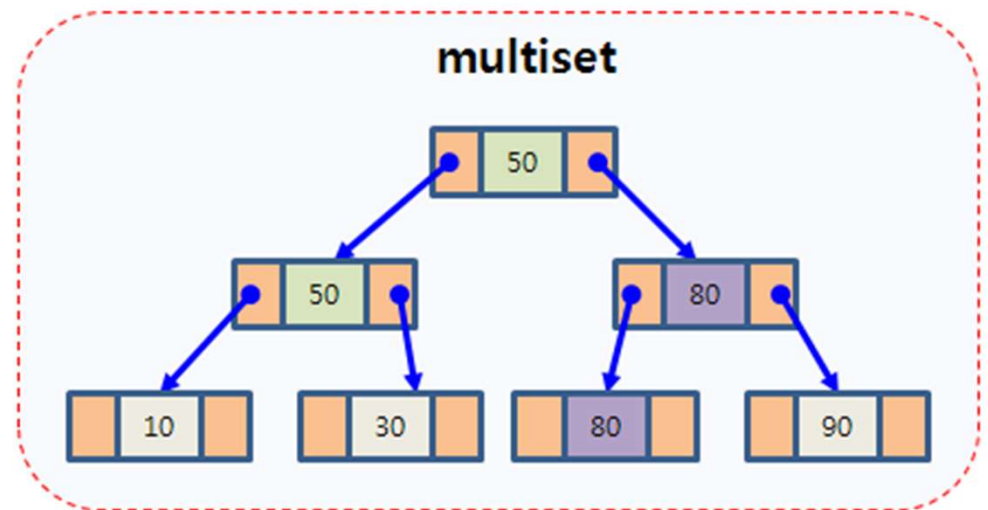
```
L'ensemble set 1 contient : 13 23 42 65 75 100
L'ensemble set 2 contient : 13 23 75
```

Conteneurs - multiset

- Les éléments sont triés selon leur valeur. Chacun des éléments peut exister plus d'une fois

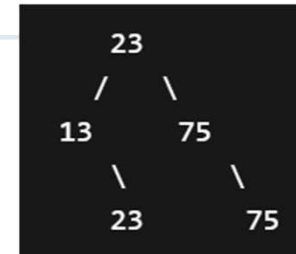
```
#include <set>
multiset<int> s;
```

```
s.insert( 50 );
s.insert( 10 );
s.insert( 30 );
s.insert( 80 );
s.insert( 50 );
s.insert( 80 );
s.insert( 90 );
```



Conteneurs - multiset

```
#include <iostream>
#include <set>
using namespace std;
int main(){
int desEntiers1[] = {75, 23, 65, 42, 13, 100}; // non ordonné
int desEntiers2[] = {75, 23, 75, 23, 13}; // non ordonné avec des doublons
multiset<int> ensemble1 (desEntiers1, desEntiers1+6); // the range (first,last)
multiset<int> ensemble2 (desEntiers2, desEntiers2+5); // the range (first,last)
cout << "L'ensemble multiset 1 contient :";
for(multiset<int>::iterator it=ensemble1.begin(); it!=ensemble1.end(); ++it)
{cout << ' ' << *it;}
cout << endl;
cout << "L'ensemble multiset 2 contient :";
for(multiset<int>::iterator it=ensemble2.begin(); it!=ensemble2.end(); ++it)
{cout << ' ' << *it;}
cout << endl;
return 0;}
```



```
L'ensemble multiset 1 contient : 13 23 42 65 75 100
L'ensemble multiset 2 contient : 13 23 23 75 75
```

Conteneurs - map

- Les éléments sont une paire formée d'une clé jumelée à une valeur.
- Les éléments sont triés selon leur clé.
- Chacune des clés ne peut exister qu'une fois

```
#include <map>
map<string, float> coll;    //Déclaration d'une map
                             //la clé est un string
                             // la valeur est un float
coll["TVA"] = 0.15;         // TVA est la clé
coll["Pi"] = 3.1416;
coll["un nombre arbitraire"] = 4983.223;
coll["zero"] = 0;
```

Conteneurs - multimap

- Même chose que map excepté qu'une clé peut exister plus d'une fois.
- On peut donc retrouver des éléments possédant la même clé mais des valeurs différentes
- Ex: dictionnaire

```
#include<iostream>
#include<map>
#include <string>
using namespace std;
```

```
int main()
{map<string, int> phoneNums;
//Insert key, value as pairs
phoneNums.insert(pair<string, int>("A",123));
phoneNums.insert(pair<string, int>("B",444));
//Insert duplicates
phoneNums.insert(pair<string, int>("A",369));
phoneNums.insert(pair<string, int>("B",567));
phoneNums.insert(pair<string, int>("A",888));
phoneNums.insert(pair<string, int>("B",999));
//Checking frequency of different keys
cout<<"Frequency of different names"<<endl;
cout<<"Number of Phones for A= "<<phoneNums.count("A")<<endl;
cout<<"Number of Phones for B = "<<phoneNums.count("B")<<endl;
cout<<"Number of Phones for S = "<<phoneNums.count("S")<<endl;
cout<<"Number of Phones for Z = "<<phoneNums.count("Z")<<endl;
return 0;}
```

```
Frequency of different names
Number of Phones for A= 1
Number of Phones for B = 1
Number of Phones for S = 0
Number of Phones for Z = 0
```

```
#include<iostream>
```

```
#include<map>
```

```
#include <string>
```

```
using namespace std;
```

```
int main()
```

```
{multimap<string, int> phoneNums;
```

```
//Insert key, value as pairs
```

```
phoneNums.insert(pair<string, int>("A",123));
```

```
phoneNums.insert(pair<string, int>("B",444));
```

```
//Insert duplicates
```

```
phoneNums.insert(pair<string, int>("A",369));
```

```
phoneNums.insert(pair<string, int>("B",567));
```

```
phoneNums.insert(pair<string, int>("A",888));
```

```
phoneNums.insert(pair<string, int>("B",999));
```

```
//Checking frequency of different keys
```

```
cout<<"Frequency of different names"<<endl;
```

```
cout<<"Number of Phones for A= "<<phoneNums.count("A")<<endl;
```

```
cout<<"Number of Phones for B = "<<phoneNums.count("B")<<endl;
```

```
cout<<"Number of Phones for S = "<<phoneNums.count("S")<<endl;
```

```
cout<<"Number of Phones for Z = "<<phoneNums.count("Z")<<endl;
```

```
return 0;}
```

```
Frequency of different names
```

```
Number of Phones for A= 3
```

```
Number of Phones for B = 3
```

```
Number of Phones for S = 0
```

```
Number of Phones for Z = 0
```



```

#include<iostream>
#include<map>
#include <string>
using namespace std;
void printer(multimap<string, int> pN)
{cout<<"\n\nMultimap printer method"<<endl;
cout<<"Map size = "<<pN.size()<<endl;
multimap<string, int>::iterator it = pN.begin();
while(it != pN.end())
{cout<<"Key = "<<it->first<<" Value = "<<it->second<<endl; it++;} }
int main()
{multimap<string, int> phoneNums;
//Insert key, value as pairs
phoneNums.insert(pair<string, int>("A",123));
phoneNums.insert(pair<string, int>("B",444));
//Insert duplicates
phoneNums.insert(pair<string, int>("A",369));
phoneNums.insert(pair<string, int>("B",567));
phoneNums.insert(pair<string, int>("A",888));
phoneNums.insert(pair<string, int>("B",999));
//Checking frequency of different keys
cout<<"Frequency of different names"<<endl;
cout<<"Number of Phones for A = "<<phoneNums.count("A")<<endl;
cout<<"Number of Phones for B = "<<phoneNums.count("B")<<endl;
cout<<"Number of Phones for S = "<<phoneNums.count("S")<<endl;
cout<<"Number of Phones for Z = "<<phoneNums.count("Z")<<endl;
//printing values
printer(phoneNums); return 0;}

```

Conteneurs dérivés

- Le standard C++ fournit quelques conteneurs supplémentaires:
 - **Piles** (stacks): conteneur manipulant ses éléments selon la politique LIFO (last-in-first-out)
 - **Files** (queues): conteneur manipulant ses éléments selon la politique FIFO (first-in-first-out)

Pile (stack)

- Les méthodes les plus importantes sont : `push(valeur)`, `pop( )` et `top( )`

```
#include <iostream>
#include <stack>
using namespace std;
int main(void)
{
    stack<int> p;
    p.push(2);
    p.push(3);
    cout << "Nombre d'éléments dans la pile : " << p.size() << endl;
    cout << "Le premier élément est : " << p.top() << endl; // accès au dernier élément
    p.pop(); // suppression du dernier élément
    cout << "Le deuxième élément est : " << p.top() << endl; // accès au deuxième
        élément
    p.pop(); // suppression du dernier élément ; maintenant la pile est vide !
    cout << "Nombre d'éléments dans la pile : " << p.size() << endl;
    return 0;
}
```

File (queue)

- Cette structure dérive de celle de **deque**, avec en plus les méthodes `push(valeur)` et `pop( )`.

```
#include <queue>
using namespace std;
int main(void)
{
    queue<int> i;
    i.push(10);
    i.push(20);
    i.push(30);
    i.push(40);
    cout << "Le premier élément est : " << i.front() << endl;
    cout << "Le dernier élément est : " << i.back() << endl;
    i.pop();
    cout << "Le premier élément est : " << i.front() << endl;
    cout << "Le dernier élément est : " << i.back() << endl;
    return 0;
}
```

Avant `i.push(40)`

`i.front()`

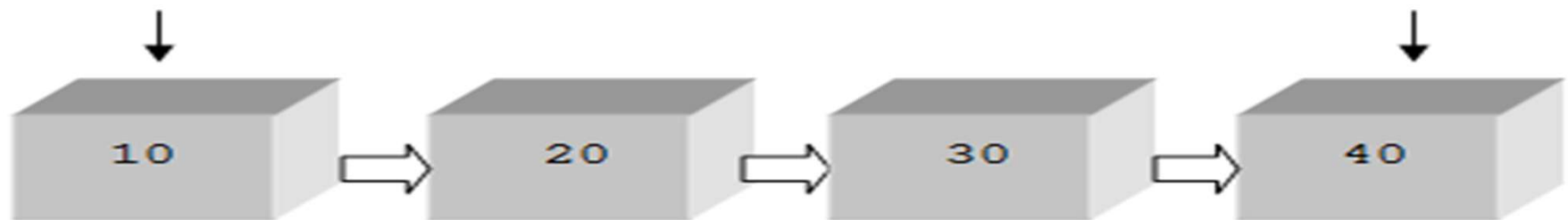
`i.back()`



Après `i.push(40)`

`i.front()`

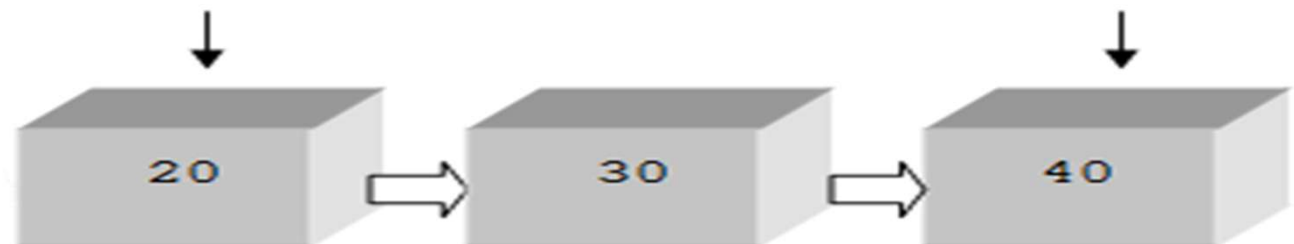
`i.back()`



Après `i.pop()`

`i.front()`

`i.back()`



Conteneurs - requis

- L'élément qui sera inclus dans un conteneur doit posséder:
 - constructeur copie
 - opérateur d'affectation (operator =)
 - destructeur
 - possiblement, un constructeur par défaut
 - possiblement, un test d'égalité (operator ==)
 - possiblement, un critère de tri (operator <, ...)

```
bool operator < (const Complex & z) const {  
    return ((re < z.re) || ((re == z.re) && (im == z.im)));  
}
```

Itérateurs

- ➡ Un **itérateur** est un objet qui permet de naviguer dans les éléments d'un conteneur
- ➡ Un itérateur est un pointeur « intelligent »
(considéré comme intelligent car le déplacement dans un arbre binaire équilibré n'est pas comme un déplacement dans une liste d'éléments contigus)
- ➡ L'itérateur fait le lien entre les conteneurs et les algorithmes

Itérateurs - opérations

- 👉 L'itérateur a une interface similaire aux pointeurs (operator*, operator++, operator==, operator!=, operator=, and operator->)
- 👉 **Opérateur *** : Retourne l'élément de la position courante
- 👉 **Opérateur ++** : Fait pointer l'itérateur à l'elt suivant
- 👉 **Opérateur ==** : Indique si 2 itérateurs pointent le même élément
- 👉 **Opérateur =** : Assigne un itérateur

Conteneurs – fonctions membres

- Tous les conteneurs offrent les mêmes fonctions de base permettant aux itérateurs d'accéder aux éléments
 - **begin()** : Retourne un itérateur représentant le début des éléments dans le conteneur
 - **end()** : Retourne un itérateur représentant la fin des éléments dans le conteneur
 - **size()** : Retourne le nombre d'éléments présents dans le conteneur
 - **empty()** : Retourne vrai si conteneur est vide
 - **erase(itérateur)**: retire l'élément se trouvant à la position indiquée par l'itérateur et retourne un itérateur sur l'élément suivant
 - **erase(itérateur, itérateur)**: retire tous les éléments à partir de la position indiquée par le premier itérateur jusqu'à la position indiquée par le second
 - **back()**: retourne le dernier élément du conteneur
 - **front()**: retourne le premier élément du conteneur
 - **pop_back()**: retire le dernier élément du conteneur
 - **push_back()**: insère un nouvel item à la fin du conteneur

Itérateurs - utilisation

- Tous les conteneurs définissent deux types d'itérateur
 - `container::iterator`
permet de naviguer en mode lecture/écriture
 - `container::const_iterator`
permet de naviguer en mode lecture seulement

```
Template <typename T>
class vector {
    public:
        class iterator {
            ...
        };
    ...
};
```

Ainsi, un itérateur pour un vecteur d'entiers sera déclaré de la manière suivante:

```
vector<int>::iterator iter;
```

Itérateurs - utilisation

```
#include <list>
// création d'une liste de caractères
list<char> coll;

for (char c = 'a'; c<='z'; c++)
    coll.push_back(c);

// impression de la liste à l'aide d'un itérateur
list<char>::const_iterator pos;

for(pos = coll.begin(); pos != coll.end(); pos++)
    cout << *pos << ' ';
```

***pos = ...; // est illégale !**

```

#pragma warning(disable:4786)
#include <map>
#include <string>
#include <iostream>
using namespace std;

void main()
{
    map< int, string > m;
    map< int, string >::iterator it;

    m.insert(map< int, string >::value_type( 2, "Deux" ));
    m.insert(map< int, string >::value_type( 3, "Trois" ));
    m.insert(map< int, string >::value_type( 5, "Cinq" ));

    cout << "Enter a Number: ";
    int num; cin >> num;
    it = m.find( num );
    if ( it != m.end() )
        cout << it->second << endl;
    else
        cout << "Element absent" << endl;
}

```

Le type d'un élément de map est une paire
pair < Key, T >

Itérateurs – catégories

- Bidirectionnel
 - peut itérer en direction avant (++) et arrière (--)
 - type d'itérateur des conteneurs `list`, `set`, `multiset`, `map` et `multimap`
- Accès par index
 - peut itérer en direction avant (++), arrière (--) et par index ([])
 - type d'itérateur des conteneurs `vector` et `deque`

Les constructeurs des Conteneurs séquentiels

- Tous les conteneurs séquentiels ont quatre constructeurs
 - `vector<T>( ) :`
 - `vector<T>(taille) :`
 - `vector<T>(taille, valeur) :`
 - `vector<T>(itérateur, itérateur) :`
- Les conteneurs `list` et `deque` ont aussi ces constructeurs

```
#include <list>

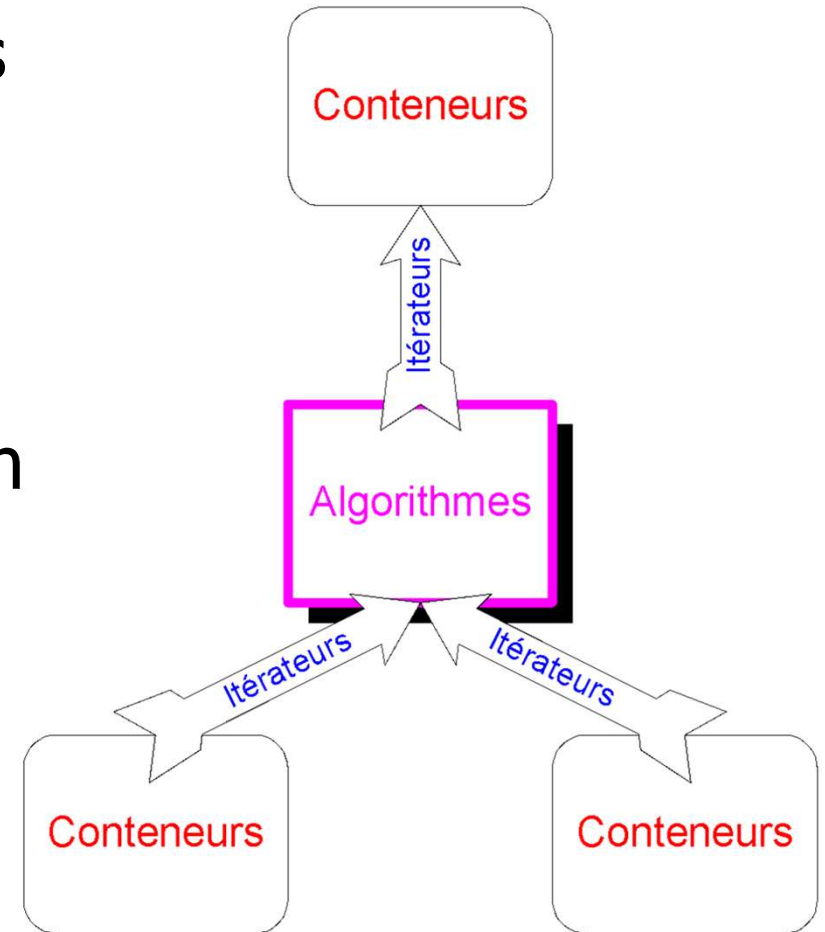
list<char> L(10, 'a');
...
```

```
#include <deque>

list<char>::iterator it;
it=L.begin(); it++; it++;
deque<char> Q(it, L.end());
```

Algorithmes

- STL fournit plusieurs algorithmes pour traiter les éléments des conteneurs
- Ces algorithmes offrent les services fondamentaux tels que recherche, tri, copie, modification
- Les algorithmes ne sont PAS des fonctions membres des classes conteneurs
- Ce sont plutôt des fonctions globales qui opèrent avec des itérateurs



Algorithmes - utilisation

```
#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;

vector<int> coll;
vector<int>::iterator pos;

// trouver le minimum et le maximum
pos = min_element(coll.begin(), coll.end());
cout << "min: " << *pos << endl;
pos = max_element(coll.begin(), coll.end());
cout << "max: " << *pos << endl;

// tri
sort(coll.begin(), coll.end());
```


Algorithmes - utilisation

// trouver l'élément avec la valeur 3

```
pos = find(coll.begin(), coll.end(),           // intervalle
           3);                                // valeur
```

```
coll.erase(pos); /*supprimer l'élément ayant la position
pos de coll */
```

// renverser l'ordre des éléments, à partir de celui ayant la valeur 3 jusqu'à la fin

```
sort(pos, coll.end());
```

// imprimer

```
for(pos = coll.begin(); pos!=coll.end()); pos++)
    cout << *pos << ' ';
```

```

#include <iostream>
#include <list>
#include <algorithm>
typedef list <char> contChar ;

contChar initC ( char * c )
{
    contChar xxx ;
    while ( *c != '\\0' ) xxx.push_back ( *c++);
    return xxx ;
}

void main ( )
{
    contChar iom = initC ("Un exemple sur les algo. génériques" );

    iom.sort ( ) ;
    iom.unique ( ) ;

    for ( contChar::iterator i = iom.begin(); i != iom.end(); i++)
        cout << *i ;

    cout << "." << endl ;

}

```

Remarque

- Pourquoi certaines fonctionnalités d'un conteneur sont disponibles comme algorithmes STL mais également comme fonctions membres d'un conteneur?
- Deux raisons:
 - a) L'algorithme ne fonctionne pas avec le conteneur en question (exemple : sort et list)
 - b) L'algorithme n'est pas assez efficace (exemple : remove_if et list)

Utilisation de fonctions dans les algorithmes

- Pour encore plus de flexibilité, certains algorithmes permettent l'usage de fonctions définies par l'utilisateur
- Les fonctions sont appelées à l'intérieur par l'algorithme

Exemple: l'algorithme `for_each`, qui appelle une fonction pour chaque élément d'un intervalle spécifié

```
// fonction usager
void print(int elem)    { cout << elem << ' '; }
void main() {
    vector<int> coll;
    ...
    for_each(coll.begin(), coll.end(), // intervalle
             print);                  // opération
}
```

Utilisation de prédicats dans les algorithmes

- Un **prédicat** est une fonction qui retourne une valeur booléenne
- Les prédicats peuvent être utilisés pour spécifier un critère de recherche ou de tri

- **Exemple:** recherche d'un nombre premier

```
bool estPremier(int nombre)           // prédicat
{
    if (nombre == 0 || nombre == 1) return true;

    int diviseur;
    for(diviseur = nombre /2; nombre %diviseur !=0; diviseur--);

    return (diviseur == 1);
}
```

Utilisation de prédicats dans les algorithmes

recherche d'un nombre premier (suite)

```
void main() {  
    list<int> coll;  
    ...  
    list<int>::iterator pos;  
    pos = find_if(coll.begin(), coll.end(),           // intervalle  
                  estPremier);                       // prédicat  
  
    if(pos != coll.end())                            // si un nombre a été trouvé  
        cout << *pos;  
}
```

La classe string de C++

- C++ a une bibliothèque `<string>`
`#include <string>`
- Namespace : `std`
- Constructeur : **`string()`, `string(char*)`, `string(string)`**
- Operator : **`+`, `+=`, `=`, `==`, `!=`, `<`, `<=`, `>`, `>=`, `[]`**
- **`substr`, `replace`, `find`, `swap`, `getline(cin, s1)`,
...**

Plus pratique que la bibliothèque string de C
(`<cstring>`)

std::string

```
#include <string> // Pour utiliser les chaînes de caractères
#include <iostream>
using namespace std ;

void main()
{
    string MaChaine="Bonjour les camarades";
    cout << "La Chaine de caractères \" <<
        MaChaine << "\" a pour taille \" <<
        MaChaine.size() << "." << endl;

    string AutreChaine("!!");
    cout << "Concaténation des deux chaines : \"
        << MaChaine + AutreChaine<< "\" << endl ;
}
```


Généralité sur les fichiers

- Vision logique des fichiers:
 - Un tableau d'octets.
 - Un pointeur de fichier indique la position courante.
- La règle générale pour créer un fichier est la suivante :
 - il faut l'ouvrir en écriture.
 - on écrit des données dans le fichier.
 - on ferme le fichier.
- Pour lire des données écrites dans un fichier :
 - on l'ouvre en lecture.
 - on lit les données en provenance du fichier.
 - on ferme le fichier.

Généralité sur les fichiers

- Il existe 2 types de fichiers :
 - les fichiers textes qui sont des fichiers lisibles par un simple éditeur de texte.
 - les fichiers binaires dont les données correspondent en général à une copie bit à bit du contenu de la RAM. Ils ne sont pas lisibles avec un éditeur de texte.
- Il existe principalement 2 bibliothèques standard pour écrire des fichiers :
 - `<stdio.h>` qui provient en fait du langage C.
 - `<fstream>` qui est typiquement C++.

Utilisation de <stdio.h>

- La fonction `FILE * fopen(const char * filepath, char * mode)`: Cette fonction permet d'ouvrir un fichier en lecture ou en écriture. Le paramètre `filepath` est un tableau de char contenant le chemin du fichier sur lequel on souhaite travailler. Le paramètre `mode` indique le mode d'ouverture de `filepath` : lecture ou écriture, texte ou binaire.

Le mode peut avoir l'une des valeurs suivantes :

- "r" (read) : lecture,
- "w" (write) : écriture, fichier créé ou écrasé s'il existait déjà,
- "a" (append) : écriture en fin de fichier existant (ajout de données).

Sur certaines plateformes (Windows par exemple), on peut y ajouter le type d'écriture (texte ou binaire), sur les autres (Linux par exemple) le type est ignoré :

- "b" : mode binaire,
- "t" : mode texte.

Enfin, on peut ajouter le signe "+" afin d'ouvrir le fichier en lecture et écriture à la fois.

Exemples :

- "r+" : ouverture en lecture et écriture,
- "wb" : ouverture en écriture binaire.

La fonction `fopen` retourne le pointeur `NULL` si l'ouverture du fichier a échoué. Dans le cas contraire, elle retourne un pointeur vers une structure `FILE`. Ce pointeur servira à écrire ou lire dans le fichier, ainsi qu'à le fermer.

- La fonction `fclose(FILE *)`: Cette fonction permet de fermer un fichier, qu'il soit ouvert en lecture ou en écriture. On passe en paramètre à cette fonction le pointeur `FILE *` fourni par la fonction `fopen(...)`.

Les fichiers binaires

- La fonction `fwrite(const void * buffer, int size, int nb, FILE * f)` : Cette fonction écrit `nb` éléments de `size` octets (soit `nb*size` octets) à partir du pointeur `buffer` (dans la RAM) vers le fichier `f` qui doit être ouvert en écriture. Il s'agit donc d'un transferts d'octets de la RAM dans un fichier.
- La fonction `fread(const void * buffer, int size, int nb, FILE * f)` : Cette fonction lit `nb` éléments de `size` octets (soit `nb*size` octets) à partir du fichier `f` (qui doit être ouvert en lecture) vers le pointeur `buffer` (dans la RAM). Il s'agit donc d'un transferts d'octets d'un fichier vers la RAM.

Exemple : écriture dans un fichier

```
#include <iostream>
#include<stdio.h>
#include <string.h>
using namespace std;
int main (void)
{ FILE * f;
  int a = 78, i, t1 [6];
  double b = 9.87;
  char c = 'W', t2 [10];
  for(i = 0; i < 6; i++)
    t1 [i] = 10000 + i;
  strcpy (t2, "AZERTYUIO");
  cout << t2 << endl;
  f = fopen ("toto.xyz", "wb");
  if (f == NULL)
    cout << "Impossible d'ouvrir le fichier en écriture !" << endl;
  else
  { fwrite (&a,sizeof(int),1,f);
    fwrite (&b,sizeof(double),1,f);
    fwrite (&c,sizeof(char),1,f);
    fwrite (t1,sizeof(int),6,f);
    fwrite (t2,sizeof(char),10,f);
    fclose (f); }
  return 0;}
```

Dans ce programme, on ouvre le fichier binaire nommé toto.xyz en écriture.

Si on a réussi à ouvrir le fichier, on y écrit un entier, un double, un char, puis un tableau de 6 entiers et finalement un tableau de 10 char.

On remarquera que pour écrire un entier il faut écrire &a pour obtenir un pointeur vers cet entier. Pour copier le tableau t1 on écrit juste t1 car t1 est déjà un pointeur vers le premier élément du tableau.

Exemple : lecture d'un fichier

```
#include <iostream>
#include<stdio.h>
#include <string.h>
using namespace std;
int main (void)
{ FILE * f;
  int a, t1 [6], i;
  double b;
  char c, t2[10];
  f = fopen("toto.xyz", "rb");
  if (f == NULL)
    cout << "Impossible d'ouvrir le fichier en lecture !" << endl;
  else
  { fread (&a,sizeof(int),1,f);
    fread (&b,sizeof(double),1,f);
    fread (&c,sizeof(char),1,f);
    fread (t1,sizeof(int),6,f);
    fread (t2,sizeof(char),10,f);
    fclose (f); }
  cout << "a=" << a << endl
    << "b=" << b << endl
    << "c=" << c << endl;
  for (i = 0; i < 6; i++)
    cout << t1 [i] << endl;
  cout << t2 << endl;
  return 0;}
```

Dans ce programme, on ouvre le fichier binaire nommé toto.xyz en lecture.

Si on a réussi à ouvrir le fichier, on lit un entier, un double un char, puis un tableau de 6 entiers et finalement un tableau de 10 char.

Les fichiers textes

- la fonction `fprintf(FILE *f, const char * format,...)` : La fonction `fprintf` permet d'écrire en mode texte dans un fichier différentes données. On n'oubliera pas de laisser un espace entre les données pour pouvoir les relire (ou un passage à la ligne).

Le paramètre `format` permet de spécifier la nature des données et des caractéristiques sur leur écriture dans le fichier (le nombre de caractères par exemple.)

Exemples de formats :

`"%d"` ==> indique un entier

`"%lf"` ==> indique un double

`"%3.7lf"` ==> indique un double avec 3 chiffres avant la virgule et 7 après.

`"%s"` ==> indique une chaîne de caractères sans espace.

- La fonction `fscanf(FILE * f, const char * format,...)` : permet de lire les données à partir d'un fichier texte en utilisant le format de données indiqué (qui est identique à `fprintf`).

Exemple : écriture dans un fichier

```
#include <iostream>
#include<stdio.h>
#include<string.h>
using namespace std;
int main (void)
{ FILE * f;
  f = fopen("A.txt", "wt");
  int a = 78, t1 [6], i;
  double b = 9.87;
  char c = 'W', t2 [10];
  for (i = 0; i < 6; i++)
    t1 [i] = 10000+i;
  strcpy (t2, "AZERTYUIO");
  if (f == NULL)
    cout << "Impossible d'ouvrir le fichier en écriture !" << endl;
  else
  { fprintf (f, "%d %lf %c ", a, b, c);
    for (i=0;i<6;i++)
      fprintf (f, "%d ", t1[i]);
    fprintf (f, "%s ", t2);
    fclose (f);  }
  return 0;}
```

Dans ce programme, on ouvre le fichier texte nommé toto.txt en écriture. Si on a réussi à ouvrir le fichier, on y écrit un entier, un double, un char, puis un tableau de 6 entiers et finalement une chaîne de caractères sans espace contenu dans un tableau de char.

Exemple : lecture d'un fichier

```
#include <stdlib.h>
#include <iostream>
#include <stdio.h>
using namespace std;
int main (void)
{ FILE * f;
  double b;
  int a, t1 [6], i;
  char c, t2 [10];
  f = fopen ("A.txt", "rt");
  if (f == NULL)
    cout << "Impossible d'ouvrir le fichier en lecture !" << endl;
  else
  { fscanf (f, "%d %lf %c ", &a, &b, &c);
    for (i = 0; i < 6; i++)
      fscanf (f, "%d ", &t1 [i]);
    fscanf (f, "%s ", t2);
    fclose (f); }
  cout << "a=" << a << endl
    << "b=" << b << endl
    << "c=" << c << endl;
  for (i = 0; i < 6; i++)
    cout << t1 [i] << endl;
  cout << t2 << endl;
  return 0;}
```

Dans ce programme, on ouvre le fichier texte nommé toto.txt en lecture.

Si on a réussi à ouvrir le fichier, on lit un entier, un double, un char, puis un tableau de 6 entiers et finalement une chaîne de caractères.

Utilisation de fstream

■ Les fichiers textes

• La classe ofstream :

- Il s'agit d'un fichier ouvert en écriture : pour créer un tel fichier il suffit d'appeler le constructeur qui a en paramètre le nom du fichier : par exemple `ofstream f("toto.txt");`.
- Pour savoir si le fichier a bien été ouvert en écriture la méthode `is_open()` renvoie `true` si le fichier est effectivement ouvert.
- Pour écrire dans le fichier on utilise l'opérateur `<<` sans oublier d'écrire des séparateurs dans le fichier texte.

• La classe ifstream :

- Il s'agit d'un fichier ouvert en lecture : pour créer un tel fichier il suffit d'appeler le constructeur qui a en paramètre le nom du fichier : par exemple `ifstream f("toto.txt");`.
- Pour savoir si le fichier a bien été ouvert en lecture la méthode `is_open()` renvoie `true` si le fichier est effectivement ouvert.
- Pour lire dans le fichier on utilise l'opérateur `>>`

Exemple : écriture dans un fichier texte

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
int main(void)
{ int a = 78, t1 [6], i;
  double b = 9.87;
  char c = 'W';
  string s;
  for (i = 0; i < 6; i++)
    t1 [i] = 10000+i;
  s = "AZERTYUIO";
  ofstream f("toto.txt");
  if (!f.is_open())
    cout << "Impossible d'ouvrir le fichier en écriture !" << endl;
  else
  { f << a << " " << b << " " << c << endl;
    for (i = 0; i < 6; i++)
      f << t1 [i] << " ";
    f << s; }
  f.close();
  return 0;
}
```

Exemple : lecture d'un fichier texte

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
int main (void)
{ int a, t1 [6], i;
  double b;
  char c;
  string s;
  ifstream f ("toto.txt");
  if (!f.is_open())
    cout << "Impossible d'ouvrir le fichier en lecture !" << endl;
  else
  { f >> a >> b >> c;
    for (i = 0; i < 6; i++)
      f >> t1 [i];
    f >> s; }
  f.close();
  cout << "a=" << a << endl
    << "b=" << b << endl
    << "c=" << c << endl;
  for (i = 0; i < 6; i++)
    cout << t1 [i] << endl;
  cout << s << endl;
  return 0;}
```

Les fichiers binaires

- La classe `ofstream` :

Pour ouvrir en écriture un fichier binaire, il suffit d'appeler le constructeur qui a en paramètre le nom du fichier et le mode d'ouverture et fixer ce deuxième paramètre à `ios::out | ios::binary`: par exemple `ofstream f("toto.xyz",ios::out | ios::binary);`.

Pour savoir si le fichier a bien été ouvert en écriture la méthode `is_open()` renvoie `true` si le fichier est effectivement ouvert.

Pour écrire dans le fichier on utilise la méthode `write((char *)buffer , int nb)` pour écrire `nb` octets dans ce fichier.

- La classe `ifstream` :

Pour ouvrir en lecture un fichier binaire, il suffit d'appeler le constructeur qui a en paramètre le nom du fichier et le mode d'ouverture et fixer ce deuxième paramètre à `ios::in | ios::binary`: par exemple `ifstream f("toto.xyz",ios::in | ios::binary);`.

Pour savoir si le fichier a bien été ouvert en écriture la méthode `is_open()` renvoie `true` si le fichier est effectivement ouvert.

Pour lire dans le fichier on utilise la méthode `read((char *)buffer , int nb)` pour lire `nb` octets de ce fichier.

Écriture dans un fichier binaire

```
#include <iostream>
#include <fstream>
using namespace std;
int main (void)
{ int a = 78, t1 [6], i;
  double b = 9.87;
  char c = 'W';
  for (i = 0; i < 6; i++)
    t1 [i] = 10000+i;
  ofstream f ("toto.xyz", ios::out | ios::binary);
  if(!f.is_open())
    cout << "Impossible d'ouvrir le fichier en écriture !" << endl;
  else
  {
    f.write ((char *)&a, sizeof(int));
    f.write ((char *)&b, sizeof(double));
    f.write ((char *)&c, sizeof(char));
    for (i = 0; i < 6; i++)
      f.write ((char *)&t1[i], sizeof(int));
  }
  f.close();
  return 0;
}
```

Lecture d'un fichier binaire

```
#include <iostream>
#include<fstream>
using namespace std;
int main (void)
{ int a, t1 [6], i;
  double b;
  char c;
  string s;
  ifstream f ("toto.xyz", ios::in | ios::binary);
  if (!f.is_open())
    cout << "Impossible d'ouvrir le fichier en lecture !" << endl;
  else
  { f.read ((char *)&a, sizeof(int));
    f.read ((char *)&b, sizeof(double));
    f.read ((char *)&c, sizeof(char));
    for (i = 0; i < 6; i++)
      f.read ((char *)&t1[i], sizeof(int)); }
  f.close();
  cout << "a=" << a << endl
  << "b=" << b << endl
  << "c=" << c << endl;
  for (i = 0; i < 6; i++)
    cout << t1 [i] << endl;
  return 0;}
```